

Table of Contents

ИНСТРУКЦИЯ ПО ВНЕДРЕНИЮ

T3-8 v0.2 — WorldTickKernel: Strict Event-Driven Model

Проект: ENIGMA v0.5.3.1.9 **Документ:** Инструкция по унификации pipeline в строгий event-driven WorldTickKernel **Версия:** v0.2 (корректированная — устранён residual dualism из v0.1) **Тип:** Техническое задание на миграцию **Цель:** построить deterministic world kernel, где player = event stream, rules = pure reducer, perception = UI projection (вне kernel)

0. Что изменилось в v0.2 (корректировки)

Корректировка 1: mode branching → ctx.interventions

v0.1 (ошибка):

```
if mode == TickMode.PLAYER:
    self._apply_player_intervention(ctx, dm_ctx, player_actions)
```

v0.2 (строго):

```
# Kernel НЕ знает слова "player". Только events.
ctx.interventions = [PlayerAction(...), PlayerAction(...)] # data-driven
# _apply_player_intervention → _apply_interventions (generic)
```

Почему: mode = PLAYER — это остаточный второй контур логики. Player — не режим, а event stream. Kernel обрабатывает interventions одинаково, независимо от источника.

Корректировка 2: RulesSubscriber = pure reducer

v0.1 (риск): RulesSubscriber мог запускать фазы, менять state напрямую.

v0.2 (строго):

```
# Rules = pure function
def handle(event, snapshot) -> Delta:
    # НЕ: запуск новых фаз
    # НЕ: доступ к прошлому тикку вне ctx
    # НЕ: долгоживущая логика
    return delta # только delta, применяется StateApplicator'ом
```

Правило: $\text{Rules} = \text{function}(\text{event}, \text{snapshot}) \rightarrow \text{delta}$. Запрещено: mutation, phase launch, cross-tick memory.

Корректировка 3: Perception вынесен из kernel

v0.1 (ошибка): perception pipeline в `_run_core_phases`.

v0.2 (строго):

kernel $\rightarrow$ produces state_{t+1}

UI $\rightarrow$ reads state_{t+1} $\rightarrow$ builds perception (outside kernel)

Почему: perception = UI layer. Включение в kernel execution order создаёт hidden coupling UI $\leftrightarrow$ simulation timing, баги «почему NPC видно до persistence».

1. Строгая модель WorldTickKernel

1.1. Математическая спецификация

WORLD TICK KERNEL

input:

state_t # текущее состояние мира
events_t # события на этом тике (вкл. player actions как InterventionEvent)

process (immutable sequence):

```
0 simulation      (LifeEngine.tick)
0.5 services     (decay, social)
1 input merge    (interventions  $\rightarrow$  delta_buffer)
2 spatial events (NPC_MOVED, PROXIMITY)
3 memory         (decay, promotion)
4 pre-decision   (topic extraction)
5 decision       (DecisionHub)
6 post-decision  (intent events)
-- rules         (PURE FUNCTION: event + snapshot  $\rightarrow$  delta)
8 drain          (handlers)
9 integration     (WorldSnapshotBuilder)
10 persistence   (commit)
```

output:

state_{t+1} # новое состояние мира

OUTSIDE KERNEL:

perception (UI reads state_{t+1} $\rightarrow$ builds projection)

1.2. Что kernel знает

✓ events (включая InterventionEvent с source="player") ✓ state (NPCState, scene_state) ✓ deterministic transitions (фазы 0-10) ✓ rules as pure reducer

1.3. Что kernel НЕ знает

✗ player (есть только InterventionEvent с source field) ✗ UI / perception / rendering ✗ mode (нет TickMode enum — interventions определяют поведение) ✗ async (kernel — pure sync function of state + events)

2. Порядок применения (v0.2)

STEP 1 → tick_orchestrator.py (A1 — InterventionEvent + ctx.interventions)
STEP 2 → tick_orchestrator.py (A2 — _run_core_phases: NO mode, NO player branching)
STEP 3 → tick_orchestrator.py (A3 — _apply_interventions: generic, не player-specific)
STEP 4 → rules_subscriber.py (B1 — pure reducer: event + snapshot → delta)
STEP 5 → game_loop/__init__.py (B2 — player actions как InterventionEvent stream)
STEP 6 → tick_orchestrator.py (B3 — execute_player_finalize deprecated)
STEP 7 → tick_orchestrator.py (C1 — affective в core_phases, без mode check)
STEP 8 → perception вынос (C2 — perception_pipeline вне kernel, в game_screen/UI)
STEP 9 → validation tests

3. PATCH SET A — Event-Driven Kernel (3 шага, ~4 часа)

STEP 1 — backend/app/services/tick_orchestrator.py (A1)

Назначение: Ввести InterventionEvent + ctx.interventions. Убрать TickMode (больше не нужен).

ДОБАВИТЬ в начало файла (или в domain/events.py):

```
@dataclass(frozen=True)
```

```
class InterventionEvent:
```

```
    """Внешнее вмешательство в мир. Источник может быть любым: player, world_scheduler, CK successor.
```

```
    Kernel НЕ знает, что такое "player". Знает только interventions.
```

```
    source: str # "player", "world_scheduler", "ck_successor", etc.
```

```
    action_type: str # "move", "attack", "talk", "directive", etc.
```

```
    payload: Dict[str, Any]
```

```
    tick: int
```

```

@classmethod
def from_player_action(cls, action_text: str, player_name: str, tick: int, **kwargs)
-> "InterventionEvent":
    """Factory для player actions (backward compat)."""
    return cls(
        source="player",
        action_type=kwargs.get("action_type", "interact"),
        payload={"text": action_text, "player_name": player_name, **kwargs},
        tick=tick,
    )

```

В _TickContext добавить поле:

```

@dataclass
class _TickContext:
    # ... existing fields ...
    # TZ-08 v0.2: interventions replace mode. Kernel не знает "player".
    interventions: List[InterventionEvent] = field(default_factory=list)

```

УБРАТЬ TickMode enum (v0.1) — больше не нужен.

ЗАМЕНИ signature execute (v0.1):

```

def execute(
    self,
    campaign_id: str,
    scene_state: dict,
    tick_number: int = 0,
    mode: TickMode = TickMode.IDLE,
    dm_ctx: Optional["DMContextDTO"] = None,
    npc_services: Optional[Any] = None,
    spatial_service: Optional[Any] = None,
    player_actions: Optional[list] = None,
) -> Union[TickResultDTO, TickPlayerResultDTO]:

```

НА:

```

def execute(
    self,
    campaign_id: str,
    scene_state: dict,
    tick_number: int = 0,
    interventions: Optional[List[InterventionEvent]] = None,
    npc_services: Optional[Any] = None,
    spatial_service: Optional[Any] = None,
    # Backward compat: dm_ctx мостируется в interventions
    dm_ctx: Optional["DMContextDTO"] = None,
) -> Union[TickResultDTO, TickPlayerResultDTO]:
    """Единая точка входа. Kernel обрабатывает interventions, не "modes".

    Если interventions пуст — idle tick (симуляция без внешнего

```

вмешательства).

Если interventions есть — каждый обрабатывается как event overlay.
"""

Backward compat: dm_ctx → interventions

```
if interventions is None:
    interventions = []
if dm_ctx is not None and not interventions:
    # Мостируем dm_ctx в InterventionEvent
    interventions = [InterventionEvent(
        source="player",
        action_type="dm_action",
        payload={"dm_ctx": dm_ctx},
        tick=tick_number,
    )]
```

STEP 2 — backend/app/services/tick_orchestrator.py (A2)

Назначение: `_run_core_phases` — immutable sequence, HET mode branching, HET player-specific логики.

ЗАМЕНИ метод `_run_core_phases` (v0.1):

```
def _run_core_phases(
    self,
    ctx: _TickContext,
    mode: TickMode = TickMode.IDLE,
    dm_ctx: Optional["DMContextDTO"] = None,
    player_actions: Optional[list] = None,
) -> None:
    # ... mode branching ...
```

НА:

```
def _run_core_phases(self, ctx: _TickContext) -> None:
    """A2-FIX v0.2: Immutable core pipeline. NO mode, NO player branching.
    Kernel обрабатывает interventions как data, не как branching.
    Все фазы вызываются всегда. Если interventions пуст — idle tick.
    """

    # Snapshot positions (для детекции spatial transitions)
    self._snapshot_positions_before(ctx)

    # Phase 0: Simulation (LifeEngine.tick)
    # B idle: LifeEngine продвигает мир
    # C interventions: LifeEngine всё равно тикает (мир не останавливается)
    self._phase_0_simulation(ctx)

    # Phase 0.5: Services (decay, social) — всегда
    self._phase_0_5_idle_services(ctx)
```

```

# Phase 1: Input merge — interventions → delta_buffer
# Это заменяет apply_intervention. Kernel применяет interventions как
events.
self._phase_1_input_merge(ctx)

# Phase 2: Event bus primary (spatial events) — всегда
self._phase_2_event_bus_primary(ctx)

# Phase 3: Memory (decay, promotion) — всегда
self._phase_3_memory(ctx)

# Phase 4: Pre-decision (topic extraction) — всегда
self._phase_4_pre_decision(ctx)

# Phase 5: Decision (DecisionHub) — всегда
# DecisionHub видит ctx.interventions (через
PerceptualKernel.recent_directive)
self._phase_5_decision(ctx)

# Phase 6: Post-decision — всегда
self._phase_6_post_decision(ctx)

# Phase 7: Rules — event-driven subscriber (PURE REDUCER, не phase)
# (вызывается автоматически через event_bus при combat events)

# Phase 8: Drain secondary (handlers) — всегда
self._phase_8_drain_secondary(ctx)

# Phase 9: Integration (WorldSnapshotBuilder) — всегда
self._phase_9_integration(ctx)

# Affective pipeline — всегда (в одном месте, не в 2)
self._run_affective_pipeline(ctx)

# Phase 10: Persistence — всегда
self._phase_10_persistence(ctx)

# NOTE: perception pipeline ВЫНЕСЕН из kernel (TZ-08 v0.2
Корректировка 3)
# UI reads state_t+1 после execute() и строит perception отдельно

```

STEP 3 — backend/app/services/tick_orchestrator.py (A3)

Назначение: _phase_1_input_merge — generic обработка interventions, не player-specific.

ДОБАВИТЬ метод (заменяет _apply_player_intervention из v0.1):

```

def _phase_1_input merge(self, ctx: _TickContext) -> None:
    """A3-FIX v0.2: Merge interventions into delta_buffer.

    Generic — обрабатывает ЛЮБОЙ source (player, world_scheduler,
    ck_successor).
    Kernel НЕ знает "player". Знает только InterventionEvent.

    Заменяет:
    - _phase_1_input (WillpowerGate для player intent)
    - _apply_player_intervention (directive handling, movement, cognitive overlay)
    """
    if not ctx.interventions:
        # Idle tick — нет interventions, WillpowerGate no-op
        return

    # NPC state injection (если interventions требуют актуального state)
    _life_engine = self._get_life_engine()
    if _life_engine:
        ctx.npc_states = _life_engine.get_npc_states(ctx.campaign_id)
        if ctx.npc_states:
            _player_entry = next((n for n in ctx.all_npcs_raw if n.get("npc_id") ==
"player"), None)
            ctx.all_npcs_raw = ctx.npc_states
            if _player_entry:
                ctx.all_npcs_raw = [n for n in ctx.all_npcs_raw if n.get("npc_id") !=
"player"]
                ctx.all_npcs_raw.append(_player_entry)

    if not ctx.all_npcs_raw:
        # Fallback: interventions могут нести npc_states в payload
        for interv in ctx.interventions:
            if interv.source == "player" and "dm_ctx" in interv.payload:
                ctx.all_npcs_raw = interv.payload["dm_ctx"].all_npcs_raw or []
                break

    # NPIC NORMALIZATION: body_state for all NPCs
    if ctx.all_npcs_raw:
        from app.models.npc_state import BODY_STATE_DISABLED
        for _npc in ctx.all_npcs_raw:
            if not _npc.get("body_state"):
                _npc["body_state"] = dict(BODY_STATE_DISABLED)

    # Process each intervention
    for interv in ctx.interventions:
        if interv.source == "player" and interv.action_type == "dm_action":
            # Player action через dm_ctx (backward compat)
            _dm_ctx = interv.payload.get("dm_ctx")
            if _dm_ctx:
                self._process_player_dm_action(ctx, _dm_ctx)
        elif interv.action_type == "directive":

```

```

        # Directive intervention (player приказывает NPC)
        self._process_directive_intervention(ctx, interv)
    elif interv.action_type == "movement":
        # Movement intervention (NPC movement intents)
        self._process_movement_intervention(ctx, interv)
    else:
        logger.debug(f"[INPUT_MERGE] unhandled intervention:
{interv.source}:{interv.action_type}")

    # WillpowerGate для player_intent (если есть)
    # (перенесено из _phase_1_input)
    if ctx.player_intent:
        self._phase_1_input(ctx)

    def _process_player_dm_action(self, ctx: _TickContext, dm_ctx:
"DMContextDTO") -> None:
        """Обработка player DM action (directive handling, cognitive overlay). """
        # (логика из v0.1 _apply_player_intervention: directive handling)
        if hasattr(dm_ctx, 'intent_resolution') and dm_ctx.intent_resolution:
            _intent_res = dm_ctx.intent_resolution
            _params = _intent_res.original_intent.parameters if
_intent_res.original_intent else None
            _sem_action = getattr(_params, 'semantic_action', None) if _params else
None
            _sem_target = getattr(_params, 'target_reference', None) if _params else
None

            if _sem_action == "MOVE" and _sem_target:
                _target_ref = _sem_target.lower()
                _is_npc_target = any(
                    _target_ref in n.get("name", "").lower() or _target_ref in
n.get("npc_id", "").lower()
                    for n in ctx.all_npcs_raw
                ) if ctx.all_npcs_raw else False

                if _is_npc_target:
                    try:
                        from app.services.social.directive_interpretation_subscriber
import DirectiveInterpretationSubscriber
                        import types
                        _target_id = getattr(_params, 'target_id', None)
                        _directive_payload = {
                            "semantic_action": _sem_action,
                            "target_reference": _target_ref,
                            "target_id": _target_id,
                            "social_pressure": 0.8,
                        }
                        _mock_event =
types.SimpleNamespace(payload=_directive_payload)
                        _directive_deltas =

```

```

DirectiveInterpretationSubscriber().handle(_mock_event, ctx.all_npcs_raw)
    if _directive_deltas:
        ctx.delta_buffer.extend(_directive_deltas)
        # COGNITIVE OVERLAY: apply deltas immediately
        for delta in _directive_deltas:
            _npc_id = delta.npc_id
            _npc_state = next((n for n in ctx.all_npcs_raw if
n.get("npc_id") == _npc_id), None)
            if not _npc_state:
                continue
            if hasattr(delta.payload, 'recent_directive_data') and
delta.payload.recent_directive_data:
                _npc_state.setdefault("perceptual_kernel", {})
["recent_directive"] = delta.payload.recent_directive_data
            if hasattr(delta.payload, 'stress_delta') and
delta.payload.stress_delta != 0:
                _npc_state.setdefault("emotion", {})[ "stress" ] =
_npc_state.get("emotion", {}).get("stress", 0.0) + delta.payload.stress_delta
            if hasattr(delta.payload, 'fear_delta') and
delta.payload.fear_delta != 0:
                _npc_state.setdefault("social_stats", {})[ "fear_of_player" ] =
_npc_state.get("social_stats", {}).get("fear_of_player", 0.1) +
delta.payload.fear_delta
            if hasattr(delta.payload, 'shock_impulse') and
getattr(delta.payload, 'shock_impulse', 0.0) > 0.5:
                _npc_state.setdefault("body_state", {})[ "shock_impulse" ] =
getattr(_npc_state.get("body_state", {}), "shock_impulse", 0.0) +
delta.payload.shock_impulse
                _npc_state.setdefault("body_state", {})[ "consciousness" ] =
max(0.0, 1.0 - delta.payload.shock_impulse)
            except Exception as e:
                logger.error(f"[CAUSALITY_CRASH]
DirectiveInterpretationSubscriber failed: {e}", exc_info=True)

    def _process_directive_intervention(self, ctx: _TickContext, interv:
InterventionEvent) -> None:
        """Generic directive intervention (не только player)."""
        # (аналогично _process_player_dm_action, но из interv.payload напрямую)
        pass # TODO: implement for CK successor directives

    def _process_movement_intervention(self, ctx: _TickContext, interv:
InterventionEvent) -> None:
        """Movement intervention (NPC movement intents)."""
        _intents = interv.payload.get("movement_intents", [])
        if not _intents:
            return
        from app.services.spatial.movement_engine import MovementEngine
        _spatial_svc = self._resolve_spatial_service(ctx)
        if _spatial_svc:
            me = MovementEngine()

```

```

me.set_spatial_service(_spatial_svc)
_tick = self.get_current_tick(ctx.campaign_id)
self._apply_drf_scoring_overlay(_intents, ctx)
changes = me.process_intents(
    _intents, _tick,
    ctx.scene_state.get("npc_positions", {}),
    campaign_id=ctx.campaign_id, scene_state=ctx.scene_state,
)
if changes and self._scene_manager:
    self._apply_with_shadow_observation(ctx, changes,
phase_label="INTERVENTION_MOVE")

```

4. PATCH SET B — Rules as Pure Reducer (3 шага, ~3 часа)

STEP 4 — backend/app/services/events/rules_subscriber.py (B1 — strict v0.2)

Назначение: RulesSubscriber = **pure reducer**. Строгие ограничения.

Создать/обновить файл:

```

"""
path: backend/app/services/events/rules_subscriber.py
Назначение: Rules agent as PURE REDUCER.
function(event, snapshot) → delta
Зависимости: app.services.events.event_bus
Основные сущности: RulesSubscriber

```

АРХИТЕКТУРНЫЙ ПРИНЦИП (TZ-08 v0.2 Корректировка 2):

Rules = pure function (event, snapshot) → delta.

ЗАПРЕЩЕНО:

- mutation state directly
- запуск новых фаз
- доступ к прошлому тикку вне ctx
- долгоживущая логика (cache, state)

РАЗРЕШЕНО:

- чтение event
- чтение snapshot (read-only)
- вычисление delta
- return delta (применяется StateApplicator'ом)

Правило: Rules = function(event, snapshot) → delta.

Любое нарушение — hidden second kernel внутри kernel.

```

"""

```

```

from __future__ import annotations

```

```

import logging

```

```
from typing import Any, Dict, Optional
from dataclasses import dataclass
```

```
logger = logging.getLogger(__name__)
```

```
@dataclass(frozen=True)
```

```
class RulesDelta:
```

```
    """Pure delta from Rules. Applied by StateApplicator, not by RulesSubscriber."""
```

```
    target_id: str
```

```
    damage: float = 0.0
```

```
    success: bool = False
```

```
    dc: int = 0
```

```
    roll: int = 0
```

```
    checks: list = None # list of check results
```

```
    def __post_init__(self):
```

```
        # frozen dataclass workaround for mutable default
```

```
        if self.checks is None:
```

```
            object.__setattr__(self, 'checks', [])
```

```
class RulesSubscriber:
```

```
    """PURE REDUCER. Rules = function(event, snapshot) → delta.
```

```
    Подписывается на combat/action events, возвращает delta.
```

```
    НЕ мутирует state. НЕ запускает фазы. НЕ имеет memory.
```

```
    """
```

```
SUBSCRIBED_EVENTS = frozenset({
    "PLAYER_ATTACKED", "ACTOR_ATTACKS", "COMBAT",
    "player_attacks", "actor_attacks",
})
```

```
    def __init__(self):
```

```
        """NO state. NO memory. NO cache. Pure function container."""
```

```
        pass # intentionally empty — pure function
```

```
    def can_handle(self, event_type: str) -> bool:
```

```
        """Может ли subscriber обработать событие."""
```

```
        return event_type in self.SUBSCRIBED_EVENTS
```

```
    def handle(
```

```
        self,
```

```
        event: Any,
```

```
        snapshot: Dict[str, Any],
```

```
    ) -> Optional[RulesDelta]:
```

```
        """PURE FUNCTION: event + snapshot → delta.
```

```
        Args:
```

event: EventDTO или dict (read-only)
snapshot: NPC states + scene_state (read-only)

Returns:

RulesDelta или None. Delta применяется StateApplicator'ом, не здесь.

ЗАПРЕЩЕНО:

- mutation of snapshot or event*
- launching phases*
- accessing past ticks*
- caching results*

```
"""
event_type = getattr(event, 'type', None) or event.get('type', '')
if not self.can_handle(event_type):
    return None

try:
    # READ-ONLY: extract data from event + snapshot
    target_id = self._extract_target(event, snapshot)
    if not target_id:
        return None

    target_npc = self._find_npc(target_id, snapshot)
    if not target_npc:
        return None

    # COMPUTE: D&D checks (pure math, no side effects)
    action_type = self._map_event_to_action_type(event_type)
    dc = self._compute_dc(action_type, target_npc)

    # DETERMINISTIC roll (seed from event id + tick)
    import hashlib
    _event_id = getattr(event, 'id', '') or str(event.get('id', ''))
    _tick = snapshot.get('tick_number', 0)
    _seed = int(hashlib.sha256(f'{_event_id}:{_tick}'.encode()).hexdigest(),
16) % 20 + 1
    roll = _seed # d20

    success = roll >= dc
    damage = self._compute_damage(action_type, roll, dc, target_npc) if
success else 0.0

    # RETURN delta (HE apply)
    delta = RulesDelta(
        target_id=target_id,
        damage=damage,
        success=success,
        dc=dc,
        roll=roll,
        checks=[{"dc": dc, "roll": roll, "success": success}],
```

```

    )

    logger.debug(
        f"[RULES_REDUCER] event={event_type} target={target_id} "
        f"dc={dc} roll={roll} success={success} damage={damage}"
    )

    return delta

except Exception as e:
    logger.error(f"[RULES_REDUCER] failed for {event_type}: {e}",
exc_info=True)
    return None

# — Pure helper methods (read-only, no side effects) —————

def _extract_target(self, event: Any, snapshot: Dict) -> Optional[str]:
    """Extract target_id from event (read-only)."""
    if hasattr(event, 'payload'):
        return event.payload.get('target_id') or
event.payload.get('target_reference')
    return event.get('target_id') or event.get('target_reference')

def _find_npc(self, npc_id: str, snapshot: Dict) -> Optional[Dict]:
    """Find NPC in snapshot (read-only)."""
    npcs = snapshot.get('all_npcs_raw', [])
    for npc in npcs:
        if npc.get('npc_id') == npc_id or npc.get('id') == npc_id:
            return npc
    return None

def _map_event_to_action_type(self, event_type: str) -> str:
    """Pure mapping."""
    mapping = {
        "PLAYER_ATTACKED": "attack",
        "player_attacks": "attack",
        "ACTOR_ATTACKS": "attack",
        "actor_attacks": "attack",
        "COMBAT": "attack",
        "combat": "attack",
    }
    return mapping.get(event_type, "player_interacts")

def _compute_dc(self, action_type: str, target_npc: Dict) -> int:
    """Pure DC computation (read-only)."""
    # Simplified D&D 5e DC
    base_dc = {
        "attack": 10,
        "player_interacts": 5,
    }.get(action_type, 10)

```

```

    # Adjust for target abilities
    abilities = target_npc.get('abilities', {})
    dex = abilities.get('dexterity', 10)
    ac = 10 + (dex - 10) // 2 # D&D 5e AC formula

    return max(base_dc, ac)

def _compute_damage(self, action_type: str, roll: int, dc: int, target_npc: Dict) ->
float:
    """Pure damage computation (read-only)."""
    # Simplified: damage = roll - dc + base
    base_damage = 4 # d8 average
    excess = max(0, roll - dc)
    return float(base_damage + excess)

def attach_to_bus(self, event_bus) -> None:
    """Подписаться на event_bus (subscription, not state)."""
    for event_type in self.SUBSCRIBED_EVENTS:
        try:
            event_bus.subscribe(event_type, self)
        except Exception as e:
            logger.warning(f"[RULES_REDUCER] subscribe failed for {event_type}:
{e}")

```

STEP 5 — backend/app/services/game_loop/_init_.py (B2)

Назначение: Player actions как InterventionEvent stream, не mode.

НАЙТИ (строки ~1107-1118) — ручной вызов Rules agent:

ЗАМЕНИ на:

```

# B2-FIX v0.2: player actions → InterventionEvent stream.
# Kernel НЕ знает "player". Знает interventions.
from app.services.tick_orchestrator import InterventionEvent

_interventions = []
if actions:
    _interv = InterventionEvent.from_player_action(
        action_text=actions[0].action_text if hasattr(actions[0], 'action_text')
    else str(actions[0]),
        player_name=actions[0].player_name if hasattr(actions[0],
'player_name') else "player",
        tick=shared_context.current_tick or 0,
        action_type=shared_context.action_type or "player_interacts",
    )
    _interventions.append(_interv)

```

```

    # Rules: pure reducer (если нужен rules_result для shared_context)
    rules_result = None
    if hasattr(self, '_rules_subscriber') and self._rules_subscriber and
_interventions:
        _combat_events = [e for e in _interventions if 'attack' in e.action_type]
        for ev in _combat_events:
            _delta = self._rules_subscriber.handle(
                event=ev,
                snapshot={"all_npcs_raw": shared_context.all_npcs_raw or [],
"tick_number": shared_context.current_tick or 0},
            )
            if _delta:
                rules_result = rules_result or {}
                rules_result.setdefault("deltas", []).append(_delta)

```

STEP 6 — backend/app/services/tick_orchestrator.py (B3)

Назначение: execute_player_finalize — deprecated, делегирует в unified execute(interventions=...).

ЗАМЕНИ весь метод:

```

def execute_player_finalize(
    self,
    player_result: TickPlayerResultDTO,
    tick_buffer: Any,
    shared_context: Any,
    actions: list,
    campaign_id: str,
    rules_result: Dict[str, Any],
    r3_direct_mode: bool = True,
) -> TickPlayerResultDTO:
    """B3-FIX v0.2: DEPRECATED. Делегирует в unified
execute(interventions=...).

```

*Раньше: отдельный pipeline (фазы 8-10 + cognitive overlay + perception).
Теперь: unified execute с interventions. Kernel не знает "player".*

Backward compat: сохранён для callers. Внутри — мостирует в execute().

```

import logging
_logger = logging.getLogger(__name__)
_logger.warning(
    "[DEPRECATED] execute_player_finalize called. "
    "Use execute(interventions=...) instead. "
    "This wrapper will be removed in TZ-08 v0.3."
)

```

Мостируем в unified execute

```

_scene_state = shared_context.scene_state if shared_context else {}
_tick = shared_context.current_tick if shared_context else 0

# Player actions → InterventionEvent
from app.services.tick_orchestrator import InterventionEvent
_interventions = []
if actions:
    _interventions.append(InterventionEvent.from_player_action(
        action_text=str(actions[0]),
        player_name="player",
        tick=_tick,
    ))

# Apply rules result deltas (если есть) как interventions
if rules_result and "deltas" in rules_result:
    for delta in rules_result["deltas"]:
        _interventions.append(InterventionEvent(
            source="rules",
            action_type="damage",
            payload={"target_id": delta.target_id, "damage": delta.damage},
            tick=_tick,
        ))

result = self.execute(
    campaign_id=campaign_id,
    scene_state=_scene_state,
    tick_number=_tick,
    interventions=_interventions,
)

return result if isinstance(result, TickPlayerResultDTO) else
TickPlayerResultDTO()

```

5. PATCH SET C — Perception Outside Kernel (2 шага, ~2 часа)

STEP 7 — backend/app/services/tick_orchestrator.py (C1)

Назначение: Affective pipeline в `_run_core_phases` (без mode check).
Perception — УБРАТЬ из kernel.

В `_run_core_phases` (STEP 2) — affective уже добавлен, perception **убран** (нет `_run_perception_pipeline`).

УБРАТЬ метод `_run_perception_pipeline` из `tick_orchestrator.py` (если был добавлен в v0.1).

STEP 8 — Perception в UI layer (C2)

Назначение: Perception pipeline — в game_screen.py / frontend, не в kernel.

Файл: backend/app/services/game_loop/__init__.py

НАЙТИ где perception pipeline вызывался (строки ~892-905 в execute_player_finalize):

ПЕРЕНЕСТИ в место, где game_screen читает world_snapshot после execute:

Создать backend/app/services/perception/perception_projector.py:

```
"""
path: backend/app/services/perception/perception_projector.py
Назначение: UI projection layer. Reads state_t+1, builds perception.
            OUTSIDE kernel. Kernel does NOT know perception.
Зависимости: app.services.perception.behavior_manifestation_service,
            app.services.perception.phenomenology_projection_service
"""

from __future__ import annotations
import logging
from typing import Any, Optional

logger = logging.getLogger(__name__)

class PerceptionProjector:
    """Reads state_t+1, builds perception. OUTSIDE kernel.

    Kernel produces state. UI reads state → builds perception.
    This class is called by game_screen / frontend, NOT by tick_orchestrator.
    """

    def __init__(self):
        from app.services.perception.behavior_manifestation_service import \
            BehaviorManifestationService
        from app.services.perception.phenomenology_projection_service import \
            PhenomenologyProjectionService
        self._manifest_svc = BehaviorManifestationService()
        self._project_svc = PhenomenologyProjectionService()

    def project(
        self,
        scene_state: dict,
        all_npcs_raw: list,
        tick: int,
    ) -> Any:
```

"""Build perception from state_t+1.

Called AFTER execute() returns. Kernel already committed state.
"""

```
try:
    _traces = self._manifest_svc.produce_traces(
        scene_state, all_npcs_raw=all_npcs_raw
    )
    _player_perception = self._project_svc.project(
        _traces, scene_state, tick=tick
    )
    return _player_perception
except Exception as e:
    logger.warning(f"[PERCEPTION_PROJECTOR] failed: {e}")
    return None
```

B game_screen.py (или game_loop/_init_.py после execute):

C2-FIX v0.2: perception OUTSIDE kernel.

После execute() — строим perception для UI.

from app.services.perception.perception_projector **import** PerceptionProjector

```
_projector = PerceptionProjector()
_player_perception = _projector.project(
    scene_state=shared_context.scene_state,
    all_npcs_raw=shared_context.all_npcs_raw or [],
    tick=shared_context.current_tick or 0,
)
if _player_perception:
    shared_context.player_perception = _player_perception
```

6. STEP 9 — backend/tests/test_tz8_v02_strict_kernel.py (NEW)

"""Tests for TZ-08 v0.2 Strict Event-Driven Kernel."""

import pytest

import inspect

from pathlib **import** Path

class TestPatchA_NoModeBranching:

"""A1-A3: NO mode, NO player branching. Interventions only."""

def test_intervention_event_exists(self):

```
    from app.services.tick_orchestrator import InterventionEvent
    ev = InterventionEvent(source="player", action_type="attack", payload={},
tick=0)
    assert ev.source == "player"
```

def test_execute_has_interventions_param(self):

```

    from app.services.tick_orchestrator import TickOrchestrator
    sig = inspect.signature(TickOrchestrator.execute)
    assert "interventions" in sig.parameters, "execute() must have interventions
param (A1)"

def test_no_tick_mode_enum(self):
    """TickMode должен быть убран (v0.2)."""
    try:
        from app.services.tick_orchestrator import TickMode
        # Если существует — проверяем что не используется в execute
        from app.services.tick_orchestrator import TickOrchestrator
        source = inspect.getsource(TickOrchestrator.execute)
        assert "TickMode" not in source, "TickMode still in execute (v0.2 not
applied)"
    except ImportError:
        pass # OK — TickMode убран

def test_run_core_phases_no_mode(self):
    from app.services.tick_orchestrator import TickOrchestrator
    sig = inspect.signature(TickOrchestrator._run_core_phases)
    assert "mode" not in sig.parameters, "_run_core_phases still has mode param
(A2 v0.2 not applied)"

def test_run_core_phases_no_player_branching(self):
    from app.services.tick_orchestrator import TickOrchestrator
    source = inspect.getsource(TickOrchestrator._run_core_phases)
    assert "if mode ==" not in source, "mode branching in core_phases (A2 v0.2
not applied)"
    assert "_apply_player_intervention" not in source, "player-specific method in
core (A2 v0.2 not applied)"

def test_phase_1_input_merge_exists(self):
    from app.services.tick_orchestrator import TickOrchestrator
    assert hasattr(TickOrchestrator, "_phase_1_input_merge"), \
        "_phase_1_input_merge missing (A3 v0.2 not applied)"

class TestPatchB_RulesPureReducer:
    """B1-B3: Rules = pure reducer."""

    def test_rules_subscriber_no_state(self):
        """RulesSubscriber НЕ должен иметь state/cache/memory."""
        from app.services.events.rules_subscriber import RulesSubscriber
        sub = RulesSubscriber()
        # Не должно быть instance attributes (кроме методов)
        instance_attrs = [k for k in sub.__dict__ if not k.startswith('_') and not
callable(getattr(sub, k, None))]
        assert len(instance_attrs) == 0, f"RulesSubscriber has state:
{instance_attrs}"

```

```

def test_rules_delta_is_frozen(self):
    from app.services.events.rules_subscriber import RulesDelta
    delta = RulesDelta(target_id="test", damage=5.0)
    # frozen dataclass
    import dataclasses
    assert dataclasses.is_dataclass(delta)

def test_handle_returns_delta_not_mutates(self):
    """handle() должен возвращать delta, не мутировать snapshot."""
    from app.services.events.rules_subscriber import RulesSubscriber
    sub = RulesSubscriber()
    event = type('Event', (), {'type': 'PLAYER_ATTACKED', 'payload': {'target_id':
'npc_1'}, 'id': 'ev_1'})()
    snapshot = {
        'all_npcs_raw': [{'npc_id': 'npc_1', 'abilities': {'dexterity': 14}}],
        'tick_number': 100,
    }
    snapshot_copy = dict(snapshot)

    delta = sub.handle(event, snapshot)

    # snapshot not mutated
    assert snapshot == snapshot_copy, "RulesSubscriber mutated snapshot (B1
v0.2 violated)"
    # delta returned
    assert delta is not None
    assert delta.target_id == 'npc_1'

```

```

class TestPatchC_PerceptionOutsideKernel:
    """C1-C2: perception outside kernel."""

    def test_no_perception_in_core_phases(self):
        from app.services.tick_orchestrator import TickOrchestrator
        source = inspect.getsource(TickOrchestrator._run_core_phases)
        assert "_run_perception_pipeline" not in source, \
            "perception in core_phases (C2 v0.2 not applied)"

    def test_perception_projector_exists(self):
        path = Path(__file__).resolve().parents[2] / "app" / "services" / "perception" /
"perception_projector.py"
        assert path.exists(), "perception_projector.py missing (C2 v0.2 not applied)"

```

```

class TestPatchA_PhaseCoverage:
    """Все фазы вызываются в _run_core_phases."""

```

```

def test_core_phases_calls_all_phases(self):

```

```

from app.services.tick_orchestrator import TickOrchestrator
source = inspect.getsource(TickOrchestrator._run_core_phases)

required_phases = [
    "_snapshot_positions_before",
    "_phase_0_simulation",
    "_phase_0_5_idle_services",
    "_phase_1_input_merge",          # заменил _phase_1_input +
    apply_intervention
    "_phase_2_event_bus_primary",
    "_phase_3_memory",
    "_phase_4_pre_decision",
    "_phase_5_decision",
    "_phase_6_post_decision",
    "_phase_8_drain_secondary",
    "_phase_9_integration",
    "_run_affective_pipeline",
    "_phase_10_persistence",
]

for phase in required_phases:
    assert phase in source, f"{phase} not in _run_core_phases (TZ-08 v0.2 not
    applied)"

```

7. СВОДКА

Patch	Step	Тип	Время
A1	InterventionEvent + ctx.interventions	Foundation	45 мин
A2	_run_core_phases (NO mode)	Foundation	1 час
A3	_phase_1_input_merge (generic)	Foundation	1.5 часа
B1	RulesSubscriber (pure reducer)	Rules	1 час
B2	game_loop: interventions stream	Rules	45 мин
B3	execute_player_finance deprecated	Rules	30 мин
C1	affective в core_phases (no mode)	Consolidation	15 мин

Patch	Step	Тип	Время
C2	perception → PerceptionProjector (outside)	Consolidation	1 час
Test	test_tz8_v02_stri ct_kernel.py	—	1 час

Итого: ~8 часов.

8. ЧТО ПАТЧ ГАРАНТИРУЕТ (v0.2)

✓ Kernel НЕ знает “player” — только InterventionEvent с source field ✓
 Нет TickMode enum — interventions определяют поведение, не mode ✓
 Нет mode branching в _run_core_phases — immutable sequence ✓ Rules =
 pure reducer (function(event, snapshot) → delta), no state, no mutation ✓
 Perception OUTSIDE kernel — PerceptionProjector читает state_t+1 после
 execute ✓ Phase 2, 3, 4 вызываются всегда (interventions пуст → idle tick)
 ✓ Affective pipeline — в одном месте ✓ execute_player_finalize deprecated
 (backward compat)

Что НЕ решается (намеренно):

- Полное удаление execute_player_finalize (backward compat wrapper)
 - Migration всех callers с dm_ctx на interventions=[...] (постепенно)
 - Async kernel (kernel — pure sync function; async только в Rules если нужно)
 - InterventionEvent для CK successor (foundation есть, но не реализован)
-

9. ПРИНЦИПЫ v0.2 (строгие)

1. **Kernel знает только:** events, state, deterministic transitions, rules as pure reducer
 2. **Kernel НЕ знает:** player, UI, perception, rendering, mode
 3. **Player = event stream**, не режим
 4. **Rules = pure function** (event, snapshot) → delta, no state/memory/mutation
 5. **Perception = UI layer**, outside kernel, reads state_t+1
 6. **No mode branching** — interventions determine behavior
 7. **No hidden second kernel** — Rules не запускает фазы, не имеет cache
-

10. ФИНАЛЬНАЯ ПРОВЕРКА

1. TZ-08 v0.2 tests

```
cd backend && python -m pytest tests/test_tz8_v02_strict_kernel.py -v
```

2. No TickMode in execute

```
python -c "  
import sys; sys.path.insert(0, 'backend')  
import inspect  
from app.services.tick_orchestrator import TickOrchestrator  
source = inspect.getsource(TickOrchestrator.execute)  
assert 'TickMode' not in source, 'TickMode in execute (v0.2 not applied)'  
print('No TickMode OK')  
"
```

3. No mode in _run_core_phases

```
python -c "  
import sys; sys.path.insert(0, 'backend')  
import inspect  
from app.services.tick_orchestrator import TickOrchestrator  
sig = inspect.signature(TickOrchestrator._run_core_phases)  
assert 'mode' not in sig.parameters, 'mode in _run_core_phases (v0.2 not applied)'  
print('No mode OK')  
"
```

4. RulesSubscriber pure

```
python -c "  
import sys; sys.path.insert(0, 'backend')  
from app.services.events.rules_subscriber import RulesSubscriber  
sub = RulesSubscriber()  
attrs = [k for k in sub.__dict__ if not k.startswith('_')]  
assert len(attrs) == 0, f'RulesSubscriber has state: {attrs}'  
print('RulesSubscriber pure OK')  
"
```

5. Perception outside kernel

```
python -c "  
import sys; sys.path.insert(0, 'backend')  
from pathlib import Path  
path = Path('backend/app/services/perception/perception_projector.py')  
assert path.exists(), 'perception_projector.py missing (v0.2 not applied)'  
print('Perception outside kernel OK')  
"
```

11. ОБНОВЛЁННЫЙ ПОРЯДОК ПРИМЕНЕНИЯ (с TZ-08 v0.2)

ФАЗА 1 — Фундамент (~24 часа)

└─ TZ-01 (срочные фиксы)	~5 ч	
└─ TZ-08 v0.2 (WorldTickKernel strict)	~8 ч	← ОБНОВЛЁННЫЙ

└─ TZ-03 (frontend↔backend контракт) ~8 ч

ФАЗА 2 — Deterministic kernel (~6 часов)

└─ Kernel Isolation ~6 ч

ФАЗА 3 — Causal chains (~20 часов)

└─ TZ-02 (каузальная труба воли) ~12 ч

└─ TZ-04 (spatial) ~6 ч

└─ TZ-05 (LLM/DM) ~7 ч

ФАЗА 4 — Cleanup (~10 часов)

└─ TZ-06 (dead code) ~7 ч

└─ TZ-07 (documentation) ~3 ч

ФАЗА 5 — Dynastic simulation (~10 часов)

└─ CK CORE LAYER ~10 ч

ИТОГО: ~70 часов = ~9 рабочих дней

КОНЕЦ ИНСТРУКЦИИ TZ-08 v0.2